Mini React Runtime Flow Guide (codex)

Mini React Todo 동작 순서 가이드 (codex 버전)

목적

이 문서는 현재 `codex-gpt-5-4-high` 프로젝트가 실제로 어떻게 동작하는지, 아래 4가지 상황별로 자세히 설명한다.

1. 웹을 처음 열었을 때
2. Todo를 새로 추가했을 때
3. 중간 Todo를 삭제했을 때
4. Todo 카드를 클릭했을 때

핵심 관점은 다음 4가지다.

- 어떤 함수가 호출되는가
- hook/state 값이 어떻게 바뀌는가
- `render -> diff -> patch -> effect`가 어떤 순서로 실행되는가
- 현재 `codex` 구현이 `feat`와 무엇이 다른가

먼저 이해할 구조

1. 상태는 `App`만 가진다

이번 과제 조건에 맞춰 state는 루트 컴포넌트 `App`에서만 관리한다.

```
export function App(): VNode {
  const [todos, setTodos] = useState<Todo[]>(initialTodos);
```

```
  const [inputText, setInputText] = useState('');
  const [editingId, setEditingId] = useState<number | null>(null);
  const [editingText, setEditingText] = useState('');
  const [filter, setFilter] = useState<TodoFilter>('all');
  const [clickCount, setClickCount] = useState(0);
  const [inspectedTodoId, setInspectedTodoId] = useState<number | null>
(initialTodos[2]?.id ?? null);
  const [debugTick, setDebugTick] = useState(0);
  const [eventLogs, setEventLogs] = useState<string[]>([
    'App가 모든 state를 관리합니다.',
    '자식 컴포넌트는 props만 받아서 렌더링합니다.',
  ]);
}
```

2. 자식 컴포넌트는 props만 받는다

`TodoInput`, `TodoList`, `TodoItem`, `TodoFooter`는 `useState`를 갖지 않고 부모가 내려준 props로만 동작한다.

```
export function TodoItem({
  todo,
  isEditing,
  isInspected,
  editingText,
  onEditInput,
  onEditCancel,
  onEditSave,
  onInspect,
  onStartEdit,
  onToggle,
  onDelete,
}: TodoItemProps): VNode {
  return (
    <li
      className={`todo-item ${todo.completed ? 'is-complete' :
''}${isInspected ? ' is-inspected' : ''}`}
      onClick={() => {
        onInspect(todo.id);
        if (!isEditing) {
          onToggle(todo.id);
        }
      }}
    >
```

3. 업데이트 기본 흐름

모든 상태 변경은 결국 아래 흐름을 탄다.

1. `setState(...)`
2. `FunctionComponent.update()`

3. `render()`로 새 VDOM 생성
4. `diff(oldVdom, newVdom)`
5. `patch(dom, ops)`
6. `scheduleEffectFlush(this)`
7. microtask에서 `flushEffects()`

관련 코드는 아래와 같다.

```
public update(): void {
  if (this.vdom === null || this.rootNode === null) {
    this.mount();
    return;
  }

  const nextVdom = this.render();
  const ops = diff(this.vdom, nextVdom);
  this.vdom = nextVdom;

  if (ops.length > 0) {
    this.rootNode = patch(this.rootNode, ops);
  }

  scheduleEffectFlush(this);
}
```

4. 이 codex 버전의 특징

이 구현은 `feat`와 비슷한 데모 UI를 갖지만, 코어는 다소 더 정돈돼 있다.

- `useState`는 같은 값이면 `update()`를 생략한다.
- `useEffect`는 `pendingEffects` 집합에 모아 두었다가 flush 한다.
- children diff는 `PATCH / INSERT / REMOVE` 구조를 사용한다.
- patch 로그도 과도한 raw 로그 대신 요약 형태로 남긴다.

시나리오 1. 웹을 처음 열었을 때

순서

1. 브라우저가 `main.ts`를 실행한다.
2. `new FunctionComponent(container, () => h(App, null))`가 만들어진다.
3. `app.mount()`가 호출된다.
4. `mount()` 안에서 `render()`가 실행된다.
5. `render()` 안에서 `setCurrentComponent(this)`가 호출된다.

6. 루트 `App()`이 처음 실행된다.
7. `useState`, `useMemo`, `useEffect`가 순서대로 hooks 배열 슬롯에 저장된다.
8. `createDom(nextVdom)`가 실제 DOM 노드를 만든다.
9. `container.replaceChildren(this.rootNode)`로 첫 DOM이 붙는다.
10. `recordPatchLogs([])`가 실행되어 현재 patch 로그는 `변경 없음` 상태가 된다.
11. `scheduleEffectFlush(this)`가 호출된다.
12. microtask에서 `flushEffects()`가 실행된다.
13. `useEffect` 본문이 `document.title`을 현재 Todo 상태에 맞게 갱신한다.

호출 함수

- `FunctionComponent.mount`
- `FunctionComponent.render`
- `setCurrentComponent`
- `App`
- `useState`
- `useMemo`
- `useEffect`
- `createDom`
- `scheduleEffectFlush`
- `flushEffects`

관련 코드

```
const app = new FunctionComponent(container, () => h(App, null));
app.mount();

public mount(): void {
  const nextVdom = this.render();
  this.vdom = nextVdom;
  this.rootNode = createDom(nextVdom);
  this.container.replaceChildren(this.rootNode);
  recordPatchLogs([]);
  scheduleEffectFlush(this);
}
```

이때 중요한 포인트

- 첫 렌더에서는 `diff()`가 아직 없다.
- 첫 화면은 `createDom()`으로 처음 생성된다.
- effect는 DOM이 붙은 뒤 실행된다.
- `codex`는 첫 렌더 이후 patch 로그를 명시적으로 초기화해 디버그 패널에 바로 보여준다.

시나리오 2. Todo를 새로 추가했을 때

예: 입력창에 텍스트를 입력하고 `추가` 버튼을 누른 경우

순서

1. 사용자가 `TodoInput`의 버튼을 클릭한다.
2. 버튼의 `onClick={() => onAdd()}`가 실행된다.
3. 부모에서 내려준 `App.handleAdd()`가 호출된다.
4. `handleAdd()`가 `inputText.trim()`으로 텍스트를 검사한다.
5. 새 Todo 객체 `{ id: nextTodoId++, text, completed: false }`를 만든다.
6. `setTodos(prev => [...prev, nextTodo])`가 실행된다.
7. `setInputText('')`가 실행된다.
8. `setInspectedTodoId(nextTodo.id)`가 실행된다.
9. `pushLog(...)`가 `eventLogs` state를 갱신한다.
10. `scheduleDebugRefresh()`가 `debugTick`을 하나 증가시키는 microtask를 예약한다.
11. 각 `setState`는 `useState`의 setter를 통해 `component.update()`를 요청한다.
12. `FunctionComponent.update()`가 호출되어 `render()`가 다시 실행된다.
13. 새 VDOM에서 Todo 리스트 끝에 새 Todo가 추가되고, input `value`는 빈 문자열이 된다.
14. `diff()`가 이전 VDOM과 새 VDOM을 비교한다.
15. 입력창에는 `UPDATE_PROPS`, Todo 리스트에는 `CHILDREN -> INSERT`가 만들어질 수 있다.
16. `patch()`가 실제 DOM에 새 Todo만 추가하고, input 값만 비운다.
17. `scheduleEffectFlush(this)` 후 `flushEffects()`가 실행되고 `document.title`도 최신 상태로 맞춰진다.

호출 함수

- `TodoInput` 버튼 `onClick`
- `App.handleAdd`
- `setTodos`
- `setInputText`
- `setInspectedTodoId`
- `pushLog`
- `scheduleDebugRefresh`
- `useState` 내부 setter
- `FunctionComponent.update`
- `diff`

- `patch`
- `scheduleEffectFlush`
- `flushEffects`

관련 코드

```
<button
  className="todo-add-button"
  type="button"
  disabled={isDisabled}
  onClick={() => {
    onAdd();
  }}
>
  추가
</button>

function handleAdd(): void {
  const trimmedText = inputText.trim();
  if (trimmedText.length === 0) {
    return;
  }

  const nextTodo: Todo = {
    id: nextTodoId++,
    text: trimmedText,
    completed: false,
  };

  setTodos((previousTodos) => [...previousTodos, nextTodo]);
  setInputText('');
  setInspectedTodoId(nextTodo.id);
  pushLog(`TodoInput -> App.handleAdd("${trimmedText}") -> todos state 업데이
        트`);
  scheduleDebugRefresh();
}
```

diff / patch 관점 해석

- 입력창: `value=""`로 바뀌므로 `UPDATE_PROPS`
- Todo 리스트: 자식이 하나 늘어나므로 `CHILDREN` 안 `INSERT`
- 전체 DOM 교체는 아니다

시나리오 3. 중간 Todo를 삭제했을 때

예: `id=3`인 Todo를 삭제한 경우

순서

1. 사용자가 해당 `TodoItem`의 삭제 버튼을 클릭한다.
2. 버튼 handler에서 `event.stopPropagation()`이 먼저 실행된다.
3. 이어서 `onDelete(todo.id)`가 호출된다.
4. 부모에서 내려준 `App.handleDelete(id)`가 실행된다.
5. `setTodos(prev => prev.filter(todo => todo.id !== id))`가 실행된다.
6. 삭제 대상이 현재 `editingId`이면 `editingId`, `editingText`도 같이 초기화한다.
7. 삭제 대상이 현재 `inspectedTodoId`이면 남아 있는 다른 Todo를 선택 상태로 바꾼다.
8. `pushLog(...)`와 `scheduleDebugRefresh()`가 호출된다.
9. `component.update()`가 실행되어 `App`이 다시 렌더링된다.
10. 새 VDOM의 Todo 리스트에서는 해당 key를 가진 항목이 빠진다.
11. `diffChildren()`는 old children과 new children를 비교하면서 key가 있는 항목은 `oldKeyToIndex`로 매칭한다.
12. 삭제된 항목은 `REMOVE`, 남아 있는 항목은 같은 key를 기준으로 `PATCH oldIndex -> newIndex` 형태로 유지된다.
13. `patch()` 안 `applyChildPatches()`는 snapshot을 기준으로 제거할 자식만 지우고, 남은 자식은 재사용하거나 필요한 nested patch만 적용한다.
14. 최종적으로 실제 DOM에서는 삭제된 카드만 사라지고, 다른 카드는 최대한 재사용된다.

호출 함수

- `TodoItem` 삭제 버튼 `onClick`
- `App.handleDelete`
- `setTodos`
- `setEditingId`
- `setEditingText`
- `setInspectedTodoId`
- `pushLog`
- `scheduleDebugRefresh`
- `FunctionComponent.update`
- `diff`
- `diffChildren`
- `patch`
- `applyChildPatches`

관련 코드

```
<button
  className="todo-action danger"
  type="button"
  aria-label={`${todo.text} 삭제`}
  onClick={(event: Event) => {
    event.stopPropagation();
    onDelete(todo.id);
  }}
>
  삭제
</button>

function handleDelete(id: number): void {
  setTodos((previousTodos) => previousTodos.filter((todo) => todo.id !==
         id));
  if (editingId === id) {
    setEditingId(null);
    setEditingText('');
  }

  if (inspectedTodoId === id) {
    const fallbackTodo = todos.find((todo) => todo.id !== id);
    setInspectedTodoId(fallbackTodo?.id ?? null);
  }

  pushLog(`TodoItem -> App.handleDelete(${id}) -> todos state에서 제거`);
  scheduleDebugRefresh();
}

const childPatches: ChildPatch[] = matchedChildren.map((match) => {
  if (match.oldIndex === undefined) {
    return {
      type: 'INSERT',
      newIndex: match.newIndex,
      node: match.node,
    };
  }

  return {
    type: 'PATCH',
    oldIndex: match.oldIndex,
    newIndex: match.newIndex,
    ops: match.ops,
  };
});
```

이때 중요한 포인트

- 현재 codex는 MOVE 패치를 쓰지 않는다.

- 대신 `PATCH oldIndex -> newIndex`와 `REMOVE`를 조합해 keyed children을 안정적으로 재배치한다.
- 즉 key는 여전히 중요하지만, 표현 방식이 `feat`와 다르다.

시나리오 4. Todo 카드를 클릭했을 때

예: 카드를 눌러 완료/미완료를 토글한 경우

순서

1. 사용자가 `li.todo-item` 전체를 클릭한다.
2. `TodoItem`의 `onClick`이 실행된다.
3. 내부에서 `onInspect(todo.id)`가 먼저 호출된다.
4. 수정 중이 아니라면 `onToggle(todo.id)`가 이어서 호출된다.
5. 부모 `App.handleInspect(id)`가 `setInspectedTodoId(id)`를 실행한다.
6. 부모 `App.handleToggle(id)`가 `setInspectedTodoId(id)`를 다시 보장하고, `setClickCount(prev => prev + 1)`를 실행한다.
7. 이어서 `setTodos(prev => prev.map(...))`가 실행되어 해당 Todo의 `completed`가 반전된다.
8. `pushLog(...)`와 `scheduleDebugRefresh()`가 실행된다.
9. `component.update()`가 호출되어 `App`이 다시 렌더링된다.
10. 새 VDOM에서는 다음 정보가 달라질 수 있다.
    - 카드 class (`is-complete`)
    - checkbox `checked`
    - footer의 남은 개수
    - hooks 패널의 `clickCount`, `todos`, memo 값
    - event log 패널 내용
11. `diff()`는 바뀐 props/text만 찾는다.
12. `patch()`는 실제 DOM의 해당 속성과 텍스트만 수정한다.
13. 마지막으로 `flushEffects()`가 실행되어 `document.title`을 최신 상태로 맞춘다.

호출 함수

- `TodoItem li.onClick`
- `App.handleInspect`
- `App.handleToggle`
- `setInspectedTodoId`
- `setClickCount`
- `setTodos`

- `pushLog`
- `scheduleDebugRefresh`
- `FunctionComponent.update`
- `diff`
- `patch`
- `scheduleEffectFlush`
- `flushEffects`

관련 코드

```
<li
  className={`todo-item ${todo.completed ? 'is-complete' : ''}${isInspected ?
' is-inspected' : ''}`}
  onClick={() => {
    onInspect(todo.id);
    if (!isEditing) {
      onToggle(todo.id);
    }
  }}
>
```

```
function handleToggle(id: number): void {
  setInspectedTodoId(id);
  setClickCount((previous) => previous + 1);
  setTodos((previousTodos) =>
    previousTodos.map((todo) =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo,
    ),
  );
  pushLog(`TodoItem -> App.handleToggle(${id}) -> completed 상태 토글`);
  scheduleDebugRefresh();
}
```

이때 중요한 포인트

- 클릭은 자식에서 시작되지만, 실제 state 변경은 부모 `App`이 한다.
- 자식 `TodoItem`은 local state 없이 props와 callback만 사용한다.
- 즉 이 화면은 `Lifting State Up`을 데모로 보여주는 구조다.

4개 시나리오 공통 요약

모든 상황에서 공통 흐름은 거의 같다.

1. 사용자 이벤트 발생

2. 자식 컴포넌트가 부모 callback 호출
3. 부모 `App`의 state 변경
4. `FunctionComponent.update()` 실행
5. `App`과 자식 함수 컴포넌트들이 다시 호출되어 새 VDOM 생성
6. `diff(oldVdom, newVdom)`로 변경점 계산
7. `patch(dom, ops)`로 실제 DOM의 필요한 부분만 수정
8. `scheduleEffectFlush()`로 effect flush 예약
9. microtask에서 `flushEffects()` 실행

즉 이 mini-react는:

- 렌더 함수 관점에서는 루트 `App`이 다시 실행되고
- DOM 관점에서는 바뀐 부분만 patch 하는 구조다

실제 React와의 차이

이 프로젝트는 과제용 mini-react이므로 실제 React와 차이가 있다.

- state를 루트 `App`에서만 관리한다
- 자식 컴포넌트는 독립 hook/state를 갖지 않는다
- Fiber, scheduling, batching 같은 고급 기능은 없다
- diff는 단순화된 tree diff + keyed child matching 구조다
- 다만 `codex` 버전은 동일 state 값일 때 update를 생략하고, effect flush도 조금 더 체계적으로 관리한다

그래도 핵심 학습 포인트는 잘 드러난다.

- 함수 컴포넌트
- hook 배열 기반 state 유지
- render -> diff -> patch -> effect 흐름
- props-only 자식 구조
- key 기반 자식 식별